

By Gregory C. Walsh and Hong Ye

Scheduling of Networked Control Systems

The defining characteristic of a networked control system (NCS) is having one or more control loops closed via a serial communication channel. Typically when the words *networking* and *control* are used together, the focus is on the control of networks, but in this article our intent is nearly inverse: not control of networks but control *through* networks. NCS design objectives revolve around the performance and stability of a target physical device rather than of the network. The problem of stabilizing queue lengths, for example, is of secondary importance. Although, on one hand, the network plays a subordinate but important role in an NCS, the feedback loops should be designed to have as small a footprint as possible in the network since the network will have many other possibly unrelated communication tasks. Integrating computer networks into control systems to replace the traditional point-to-point wiring has enormous advantages [1], including lower cost, reduced weight and power, simpler installation and maintenance, and higher reliability. As a consequence, NCSs are now common. For example, a typical new automobile has two controller area networks (CANs), a high-speed one in front of the firewall for engine, transmission, and traction control and a low-speed one for locks, windows, and other devices. In this article, in addition to introducing networked control systems, we demonstrate how dispensing with queues and dynamically scheduling control traffic improves closed-loop performance.



©2000 Image 100 Ltd.

In addition to automobiles, NCSs are also found in manufacturing plants, aircraft, HVAC systems, and many other contexts. Serial communication networks are used to exchange information and control signals between spatially distributed system components such as supervisory computers, controllers, and intelligent I/O devices (e.g., smart sensors and actuators). Standard serial communication technologies may be adapted to this networked control system context, from multi-drop RS-485 or daisy-chained RS-232 [2]-[4] (Smart Motors from Animatics) to Ethernet and wireless extensions such as IEEE 802.11 [5]. In addition, specialized network protocols have been developed, including CAN for automotive and industrial automation (DeviceNet from Allen Bradley and Smart Distributed System from Honeywell), BACNet for building automation [6], and Fieldbus (World FIP or Profibus) [7]-[9] for process control. The specialized networks are supported by a corresponding investment at the device level; for example,

Walsh (gwalsh@eng.umd.edu) and Ye are with the Department of Mechanical Engineering, University of Maryland, College Park, MD 20742, U.S.A.

CAN devices range from stand-alone CAN interface chips such as Intel's 82527 (see Fig. 1 for the authors' application), Microchip's MCP2510, and Philips' SJA1000 to microcontrollers with an integrated on-chip CAN interface, such as Motorola's 68HC05, Siemens' (or Infineon's) SAB-C167CR, and Texas Instruments' TMS320C24x series DSP.

The serial communication channel, which multiplexes signals from the sensors to the controller and/or from the controller to the actuators, serves many other uses besides control (see Fig. 2). Each of the system components connected directly to the network is denoted a physical node. Logical subdivision is also common, so, for example, although the high speed CAN network in an automobile might have 50 logical nodes, physically only a half a dozen devices may be connected. Control networks are typically local area networks, and while hierarchical collections of networks are found, control loops are closed locally. Important exceptions to this rule exist. For example, in teleoperation systems, supervisory control of discrete-event systems, and sensor networks control loops are often closed over wide area networks. A control network should have at least three nodes; with two nodes there is no reason to use a network. Plant outputs from different nodes are often coupled, and outputs on different time scales are also found. Congestion is a common issue.

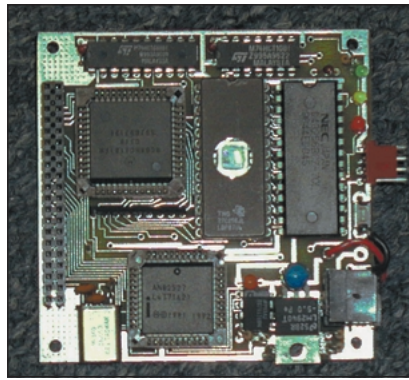


Figure 1. A networked control system smart node designed by the authors circa 1996. The card is composed of a Motorola 68HC11 with an Intel 82527 CAN interface chip.

In summary, designers choosing to use a networked control system architecture are motivated not by performance, but by cost, maintenance, and reliability gains. The use of serial communication networks in a control system is such a clear win that it is now widely supported from the device to the system level. The next section outlines some application issues with NCSs and narrows the discussion to scheduling. Scheduling control network traffic is the focus of the following section. The last section reports simulation and physical experiments.

Application Issues

Control networks differ from data networks in important ways, such as having short, frequent packets with real-time requirements. Consider the generic NCS problem in Fig. 2, where several sampled continuous-time (vector-valued) outputs, data records, are sent through a single serial communications channel and reconstructed by a smart actuator, which then uses the resulting data image to compute the control action. The computation may be carried out remotely and the commands also sent over the network, although this is not shown in the figure. A control network impacts the closed-loop performance by creating differences between the data records and their associated remote images, and the performance of a well-designed NCS degrades gracefully in the presence of congestion on the network. In a net-

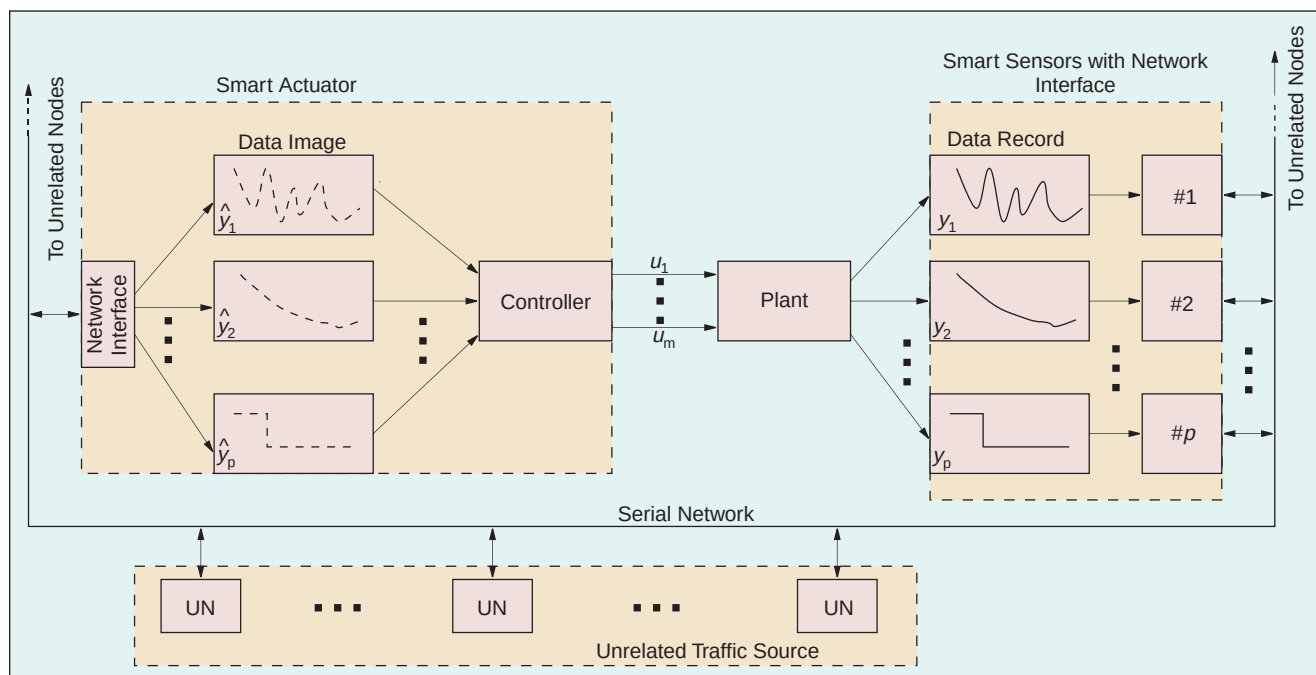


Figure 2. The essential NCS problem. The data record is located at the smart sensor and the data image at the smart actuator. Although in the figure only the measured plant outputs $y(t)$ are transmitted over the network, plant control signals $u(t)$ may also be so transmitted.

worked control system, we are interested in the signals that are carried by the bit streams, not the bit streams themselves. Issues central in data networks, such as the amount of data and the data rates, are secondary in network control.

If the network speed is high and the traffic sparse, the effect of inserting such a network into the feedback loop is that of creating a small, randomly varying time delay between the records and their images. This approach has many merits. First, the network may be treated abstractly, and hence the interface between the control system and the network can take place at a high level of the open systems interconnect (OSI) [10] model, with the associated benefits of robustness and flexibility. In addition, because the impact on control design methodology is minor, standard techniques may be applied without considering the network. This highly desirable approach is supported by several analytic results [11]–[13].

In the presence of congestion, however, some weaknesses of the approach are exposed. Many sources of delay on a control network do not pose difficulties for most applications. For example, the sampling and transmission time on a slow bit-synchronized network like CAN is less than 100 μ s; however, the amount of time a packet spends waiting to be transmitted can be far greater. This problem is exaggerated by data queues at the sensor, as we show in the following example.

Consider a SISO system, Feedback's process trainer PT326, configured as an NCS with one-packet transmission (that is to say, all sensor signals are sent with one packet and are assumed to be sampled simultaneously) through a CAN network. A Smith predictor combined with a PI controller was designed without considering the network. The network traffic is modeled as a Poisson arrival process. The smart sensor samples the output of the plant periodically and places the resulting data record into a FIFO queue. The hardware for this experiment can be seen in Fig. 8. The sensor sampling period must be larger than the average transmission time interval, otherwise the finite sensor queue will overflow. The statically scheduled policy employing a queue is compared to our try-once-discard (TOD) policy in Fig. 3. The TOD policy, which has no queue, discards data if the network is unavailable.

The transport delays in an NCS are not physical: network speeds are such that the data transmission time is very small. Simply removing the queue, which is possible since the network is local, dramatically improves the congested behavior of the networked control system. The example serves to point out a fundamental difference between data and control networks: in a control network, minimizing the ℓ_∞ norm of the difference between a data record and its associated image is more important than exactly reproducing the data image at the controller node, as would be done for a normal data network using queues and retransmissions.

The problem of congestion immediately suggests the application of information theoretic tools, but closer examina-

tion of the network protocol quickly reveals this to be a low-yield enterprise because packets on a control network are frequent and have small data segments compared to their headers. For example, a CAN II packet with a single 16-bit data sample has fixed 64 bits of overhead associated with identifier, control field, CRC, ACK field, and frame delimiter, resulting in 25% utilization, and this utilization can

The transport delays in an NCS are not physical: network speeds are such that the data transmission time

never exceed 50% (data field length is limited to 64 bits). Hence the important and general results on control through limited communication channels [14], [15] are not immediately useful in the NCS context because the overhead required to send a single packet is not reduced with the size of the data segment. Even if the sizes of the data segments were halved using some minimum-attention reasoning, the benefits would be almost unnoticeable, especially on an Ethernet or IEEE 802.11-based control network where the portion of the packet devoted to data is less than 2%.

Sending fewer packets, perhaps sending more important packets at the expense of others, has far more potential, and for this reason the problem of scheduling the real-time network traffic has captured the attention of our research group [5], [16]–[18] and many others [19]–[22]. These papers generally address the problem of efficiently using the finite bus capacity (also called the network resource) while

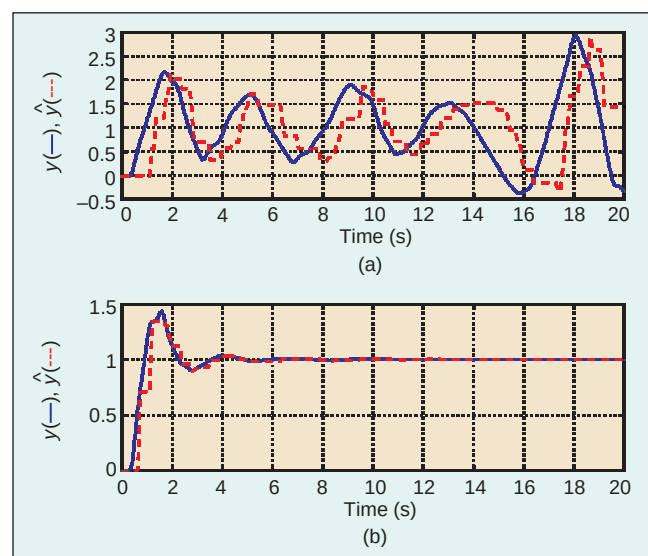


Figure 3. Comparison of the NCS with and without queue (same random transfer interval distribution $\tau = 300$ ms). Admitting packet loss yields better performance. (a) Queued NCS, $\tau = 0.30$, $T_p = 0.32$, no overflow; (b) NCS without queue, $\tau = 0.30$.

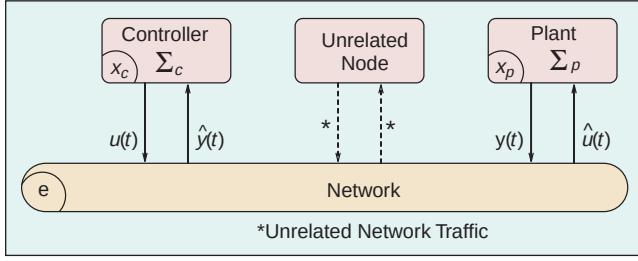


Figure 4. Configuration of a networked control system.

ignoring other important issues such as maintaining good closed-loop control system performance. In the next section, a new protocol for scheduling real-time control network traffic is introduced and compared with a commonly used static scheduler. The scheduling analysis focuses on combining the two aspects mentioned earlier.

Scheduling Analysis

Notions of fairness arise when proposing new scheduling algorithms as we do in this section. When the communication channel is allocated before runtime and the resulting schedule is not deviated from, we label the schedule *static*. In the case of fixed intervals between control system events, stability may be guaranteed [2]. Static schedules can be implemented using token rings or polling, among other methods. Static schedulers guarantee fairness but tend to be brittle in application because unanticipated events (generated externally or by some failure) simultaneously load the network and generate important changes in plant outputs. That is to say, alarm as well as feedback data are usually sent over the network. Not only is the network resource scarce because of alarms and commands, but also some of the smart sensors have important information to convey. If the schedule is static, then luck is required to weather the event gracefully. When the network resource is scarce, a dynamic scheduler deliberately starves some information sources over others. Good dynamic scheduling is system objective aware and very unfair. In this section, we propose and analytically validate a dynamic scheduler.

The NCS model considered is shown in Fig. 4. It consists of three main parts: the plant $\Sigma_p(A_p, B_p, C_p, 0)$ with state $x_p \in \mathbb{R}^{n_p}$ and output $y \in \mathbb{R}^{n_y}$; the controller $\Sigma_c(A_c, B_c, C_c, D_c)$ with state $x_c \in \mathbb{R}^{n_c}$ and output $u(t) \in \mathbb{R}^{n_u}$; and the network with state $\hat{n}(t) = [\hat{y}(t), \hat{u}(t)]^T$, consisting of the most recently reported versions of $u(t)$ and $y(t)$. Without loss of generality, we have assumed $D_p = 0$. Outputs measured locally at an actuator can be incorporated directly into the controller and do not require treatment in our model. If such outputs are needed elsewhere, the actuator node can also be considered a smart sensor. Because of the network, only the reported output $\hat{y}(t)$ is available to the controller and its prediction processes; similarly, only $\hat{u}(t)$ is available to the actuators on the plant. Commonly used networks support broadcast, hence $\hat{n}(t)$ is globally known, and in such a case the controller itself may be physically distributed.

To focus on the effect of network competition on the stability of an NCS, we make the following assumptions. We assume the control law is designed in advance without considering the presence of the network, which adds to the conservativeness of the result since some controller-plant pairs can be destabilized with very small time delays. Designing a controller that takes the network into account is a rich and interesting problem of future study. The controller dynamics are considered continuous, and sampling delay is ignored because the access interval of the NCS to the network is much larger than the processing period of the controller and smart sensors. Once access to a particular sensor node is granted, data is assumed to be transmitted instantly, since most of the NCS is connected by a local area network with a very high data rate. The communication medium is error free based on the higher reliability offered by many error detection and correction technologies used in digital communication. No observation noise exists. All matrices in this article have compatible dimensions, and the standard Euclidean norm will be used unless noted otherwise.

We label the network-induced error $e(t) := \hat{n}(t) - [y(t), u(t)]^T$ and the combined state of the controller and plant $x(t) = [x_p(t), x_c(t)]^T$. The state of the entire NCS is given by $z(t) = [x(t), e(t)]^T$, and between transmission instances the dynamics of the NCS can be summarized as

$$\dot{z}(t) = \begin{bmatrix} \dot{x}(t) \\ \dot{e}(t) \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} x(t) \\ e(t) \end{bmatrix} \quad (1)$$

where

$$\begin{aligned} A_{11} &= \begin{bmatrix} A_p + B_p D_c C_p & B_p C_c \\ B_c D_c & A_c \end{bmatrix}, \\ A_{12} &= \begin{bmatrix} B_p D_c & B_p \\ B_c & 0 \end{bmatrix}, \\ A_{21} &= -\begin{bmatrix} C_p & 0 \\ 0 & C_c \end{bmatrix} A_{11}, \\ A_{22} &= -\begin{bmatrix} C_p & 0 \\ 0 & C_c \end{bmatrix} A_{12}. \end{aligned} \quad (2)$$

Define the matrix A such that $\dot{z}(t) = Az(t)$. Any prediction or filtering process can be used to improve the estimate of $\hat{n}(t)$. Such predicting and filtering will add extra states and dynamics, which we incorporate in matrices A_{21} and A_{22} .

Without a network, $e(t) = 0$, and hence the dynamics reduce to $\dot{x}(t) = A_{11}x(t)$. It is assumed that the controller has been designed ignoring the network, hence A_{11} is Hurwitz. Consequently, there exists a unique symmetric positive definite matrix P such that

$$A_{11}^T P + P A_{11} = -I. \quad (3)$$

Define the constants $\sigma_1 = \lambda_{\min}(P)$ and $\sigma_2 = \lambda_{\max}(P)$. Since we are modeling the network as a perturbation on the system, choosing the right-hand side of (8) equal to $-I$ is desirable

for maximizing the tolerable perturbation bound (see [23, p. 206]). Such an analysis approach ignores the strong structure of $e(t)$, and our results are conservative.

The behavior of the network-induced error $e(t)$ is mainly determined by the architecture of the NCS and the scheduling strategy. In the special case of one-package transmission, there is only any one node transmitting control data on the network; therefore, the entire vector $e(t)$ is set to zero at each transmission time. For multiple nodes transmitting measured outputs $y(t)$ and/or computed inputs $u(t)$, the transmission order of the nodes depends on the scheduling strategy chosen for the NCS. In other words, the scheduling strategy decides which components of $e(t)$ are set to zero at the transmission times. Both dynamic and static schedulers will be analyzed for NCS implementation.

A dynamic scheduler determines the network allocation while the system runs. Unlike dynamically scheduling processor time in real-time control, however, the information needed to decide which node should be granted access to the network is not centrally located. Several methods for distributed decision making are possible; we will focus on one employing the bit-wise arbitration technology of CAN. For dynamic scheduling, each node will estimate how important the local data is and encode this measure into its identifier. In this article, we use a weighted absolute value of the error. The node with the greatest error wins the right to transmit. We label our technique “maximum-error-first with try-once-discard” (MEF-TOD) because if a data packet fails to win the competition for network access, it is discarded and new data is used the next time.

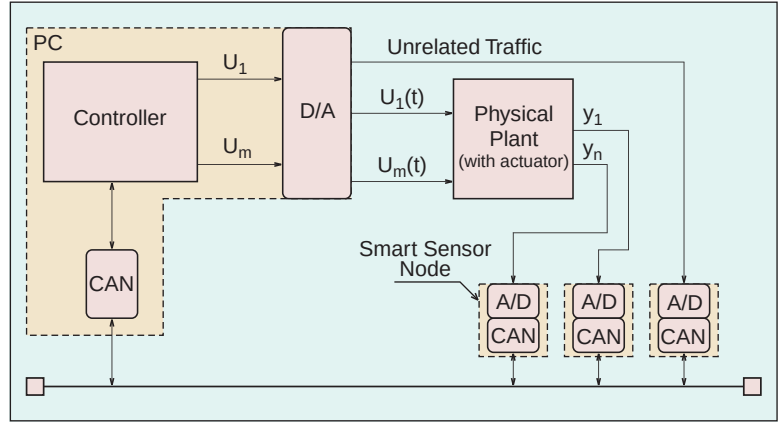


Figure 5. Configuration of the experimental system.

Without loss of generality, assume there are p nodes competing, each of which may be associated with one or multiple plant inputs and outputs. In the MEF-TOD protocol, the priority level of each node's message is proportional to the norm of $e_i(t)$, which is a k -dimensional subvector of $e(t)$, with $k \in [1, n_r + n_q]$ representing the number of plant or controller outputs transmitted by node i . The (normalization) weights assigned to error signals are assumed already built into the output matrix. At every transmission time, the node with the highest priority (or greatest weighted error) gets transmitted. If two or more messages have equal priority, a prespecified ordering of the nodes encoded in the least significant bits of the identifier resolves the collision.

Today, static scheduling is the default methodology. Although the schedule is fixed, some nodes may be granted access multiple times before others get any access. If a transmission pattern is of length N , every N consecutive visits form a repeated cycle. In one cycle, all nodes are vis-

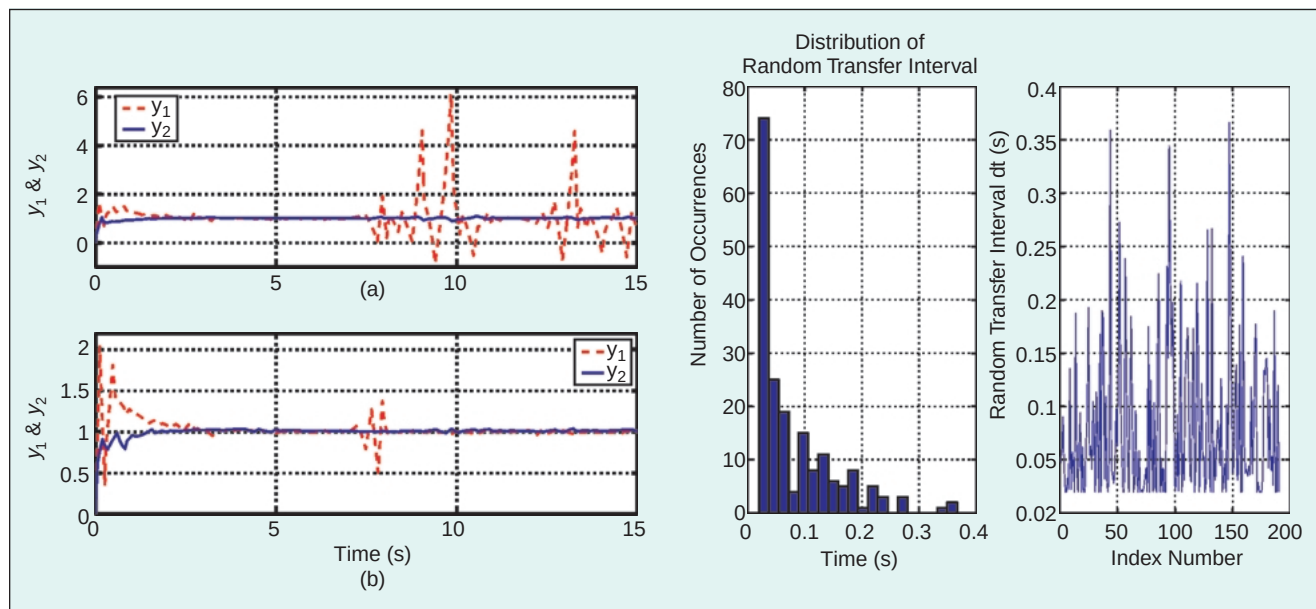


Figure 6. Comparison of different protocols under one run (with batch reactor as plant) with associated distribution of the random transfer interval. (a) Step response with token-passing protocol ($\tau = .08$ s); (b) step response with TOD protocol ($\tau = .08$ s).

ited at least once. N is then called the periodicity of the static scheduler.

To bound the amount of time between transmission events, we introduce the notion of a maximum allowable transfer interval, τ . The maximum allowable transfer interval is a deadline; if a transmission of control data takes place at time t_0 , then another one must occur within the time interval $(t_0, t_0 + \tau]$. The scheduler, dynamic or static, will determine which channel is transmitted at that time. In addition, over intervals of time of length τ the growth in error $e(t)$ is assumed to be bounded by β . The value of β depends on the system characteristics and initial conditions. The following two lemmas characterize the scheduling algorithms.

Lemma 1 (Dynamic Scheduler Error Bound): Given a dynamic (MEF-TOD) network scheduler starting at time t_0 with p nodes competing, maximum allowable transfer interval τ , and maximum growth in error in τ seconds strictly bounded by $\beta \in (0, \infty)$, then for any time $t \geq t_0 + p\tau$, $\|e(t)\| < \beta p(p+1)/2$.

Proof: There are at least p transmissions in the interval $[t - p\tau, t]$. Let $t_1, \dots, t_p$ be the last p transmission times, with $t_0 \leq t_1 < \dots < t_{p-1} < t_p \leq t$, $t_1 \geq t - p\tau$, and let $i_1, \dots, i_p$ be the nodes that were transmitted at those times, respectively. Suppose the first $k \in [1, p]$ nodes being transmitted are distinct, and the $(k+1)$ th transmitted node was also transmitted earlier, say at time $t_l, l \in [1, k]$. Then $\|e_{i_j}(t)\| < j\beta$ for $j = 1, \dots, k$. Since node i_l was transmitted both at t_l and t_{k+1} , we have $\|e_{i_l}(t_l^-)\| < (k+1-l)\beta$, with t_l^- denoting the instant right before transmission. By the construction of the dynamic scheduler (TOD), at t_l transmission time, node i_l has the greatest error. As a consequence, $\|e_j(t_l)\| \leq \|e_{i_l}(t_l^-)\| < (k+1-l)\beta$ and $\|e_j(t)\| < (k+1)\beta$ for all $j \neq i_l, j \in [1, p]$. Thus, $\|e(t)\| \leq \sum_{j=1}^p \|e_{i_j}(t)\| < \sum_{j=1}^k j\beta + (p-k)(k+1)\beta$.

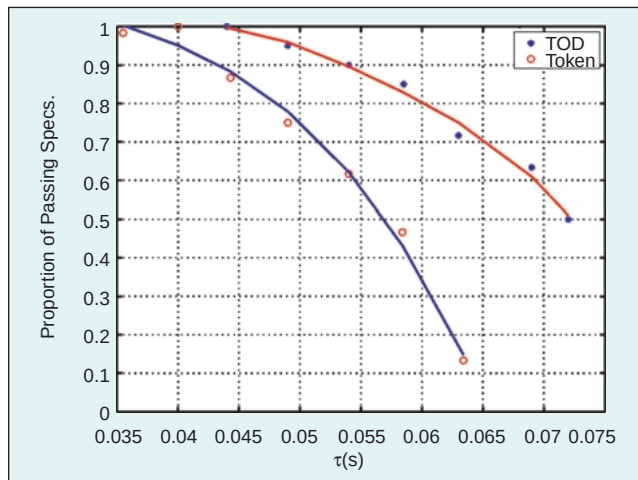


Figure 7. Comparison of the static scheduler (token passing) and the dynamic scheduler (TOD) with batch reactor as plant (60 runs per point).

Since $\max_{1 \leq k \leq p} \{\sum_{j=1}^k j\beta + (p-k)(k+1)\beta\} = \beta p(p+1)/2$, when $k = p-1$ or p , we have the worst case error bound for the dynamic scheduler $\|e(t)\| < \beta p(p+1)/2$.

Lemma 2 (Static Scheduler Error Bound): Given a static network scheduler starting at time t_0 , with integer periodicity $p = N$, maximum allowable transfer interval τ , and maximum growth in error in τ seconds strictly bounded by

A dynamic scheduler determines the network allocation while the system runs.

$\beta \in (0, \infty)$, then for any time $t \geq t_0 + p\tau$, $\|e(t)\| < \beta p(p+1)/2$.

Proof: The integer periodicity p allows at most p nodes to compete. Assume there are K nodes, $K \in [1, p]$. This also implies that each node is visited at least once during every p consecutive transmissions.

At least one cycle (or p transmissions) is completed during the interval $[t - p\tau, t]$. Let $t_1, \dots, t_p$ be the last p transmission times with $t_0 \leq t_1 < \dots < t_p \leq t$, $t_1 \geq t - p\tau$, and let $i_1, \dots, i_p$ be the nodes that were transmitted at those times, respectively. Then $\|e_{i_m}(t)\| < m\beta$ for $m = 1, \dots, p$. Since the set $\{1, \dots, K\} \subseteq \{i_1, \dots, i_p\}$, we have $\|e(t)\| \leq \sum_{j=1}^K \|e_{i_j}(t)\| < \sum_{j=1}^p j\beta = \beta p(p+1)/2$. Thus, for any time $t \geq t_0 + p\tau$, $\|e(t)\| < \beta p(p+1)/2$.

The worst case error bound of the dynamic scheduler is the same as that of a special case of the static scheduler (i.e., all p nodes are visited equally). The bound is conservative for both scheduling algorithms because τ represents a deadline. For same transmission time distributions, however, the error bound for the dynamic scheduler will be better than that for the static scheduler because the former grants access to the node with the greatest error.

The general stability condition of an NCS with either a static or a dynamic (MEF-TOD) scheduling algorithm is presented in the following theorem (refer to [16] for a detailed proof).

Theorem 1 (Stability of NCS): Given a networked control system whose continuous dynamics are described by (1) with p nodes of sensors operating under MEF-TOD or static sched-

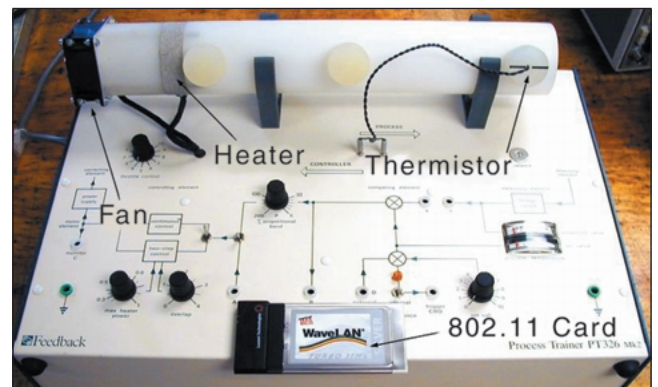


Figure 8. The Feedback PT326 process trainer is shown with one of the IEEE802.11 PC cards.

uling, and a maximum allowable transfer interval that satisfies

$$\tau < \min \left\{ \tau_0, \frac{\tau_0}{2\sigma_2 \|A\| \sqrt{\frac{\sigma_2}{\sigma_1}}} \right\}$$

where

$$\tau_0 = \left(4p(p+1) \|A\| \sqrt{\frac{\sigma_2}{\sigma_1} + 1} \right)^{-1},$$

then the networked control system is globally exponentially stable.

Proof Sketch: Initially no assumption can be made about the initial conditions of the error $e(t)$. Using the initial condition $z(t_0)$ and the Lipschitz constants of the system equations, a growth bound β can be calculated for the first $p\tau$ seconds from the bound on the growth of $z(t_0)$. At transmission times, the error $e(t)$ can only decrease, so it may be shown that β applies to the entire interval $[t_0, t_0 + p\tau]$. At time $t_0 + p\tau$ the lemmas can be employed to compute a smaller bound on $e(t)$. Once it is verified that this bound holds beyond the instant $t_0 + p\tau$, a Lyapunov argument shows that, for a $t_1 > t_0 + p\tau$, $\|z(t_1)\| < \rho \|z(t_0)\|$ for $\rho \in [0, 1]$. At time t_1 , the argument may be repeated, generating a smaller β and hence bound on $e(t)$. By induction one concludes $\|z(mt_1)\| < \rho^m \|z(t_0)\|$.

Simulations and Experiments

Two example systems, an unstable batch reactor and a dryer, are used to support the analysis above and verify the reasonableness of the assumptions. The basic configuration is shown in Fig. 5. The controller is located near the actuator,

and the control signals can be sent to the plant directly. The controller is implemented by a PC with a CAN interface and D/A converter. The smart sensor nodes with CAN interface and A/D function are distributed along the network; all plant outputs are sampled by the smart sensors and transmitted to the controller via the controller area network. The dryer is a commercial process trainer PT326 provided by Feedback, Inc. For safety reasons, the physical plant dynamics of the batch reactor are simulated via the computer, although the network is used to move information.

Batch Reactor Experiment

The unstable batch reactor (see [24, p. 62]) is a coupled two-input, two-output NCS. Based on the linearized process model (see (5)), a proportional-plus-integral controller (see (6)) is designed in advance to stabilize the feedback system and achieve good performance.

$$\dot{x} = \begin{bmatrix} 138 & -0.2077 & 6.715 & -5.676 \\ -0.5814 & -4.29 & 0 & 0.675 \\ 1.067 & 4.273 & -6.654 & 5.893 \\ 0.048 & 4.273 & 1.343 & -2.104 \end{bmatrix} x + \begin{bmatrix} 0 & 0 \\ 5.679 & 0 \\ 1.136 & -3.146 \\ 1.136 & 0 \end{bmatrix} u,$$

(4)

$$y = \begin{bmatrix} 1 & 0 & 1 & -1 \\ 0 & 1 & 0 & 0 \end{bmatrix} x,$$

(5)

$$K(s) = \begin{bmatrix} 0 & \frac{2s+2}{s} \\ \frac{-5s-8}{s} & 0 \end{bmatrix}.$$

(6)

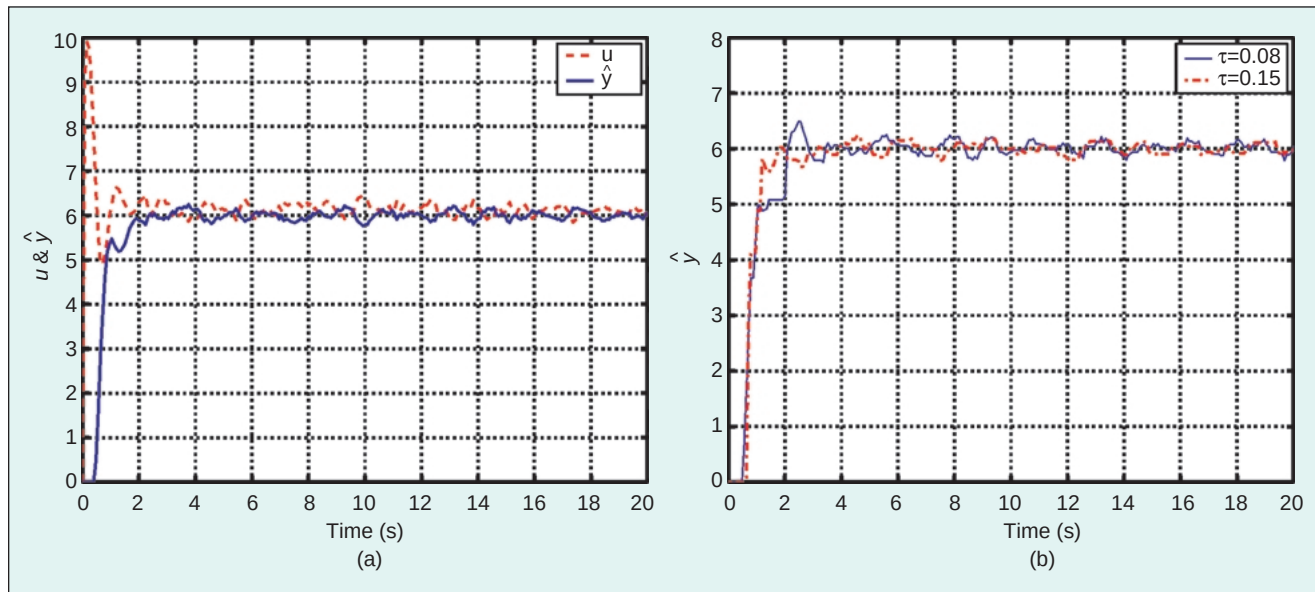


Figure 9. Step response of the PT326 under a constant transfer interval of $T_s = 80$ ms and under a random transfer interval with plant delay estimated at 400 ms using a CAN network. (a) PT326 with fixed transfer interval $T_s = 0.08$ and estimated delay $= 4T_s$; (b) random transfer interval and estimated delay $= 5T_s$.

Only the system outputs y_1 and y_2 need to be sampled and transmitted to the controller via the network, each with its associated smart node (a Phytec miniMODULE-167CAN board). In the experiment, the network is placed between the outputs of the plant and the inputs of the controller. A Poisson process with mean $1/\tau$ models the packet arrival events.

Designers choosing to use a networked control system architecture are motivated not by performance, but by cost, maintenance, and reliability.

One run of the system under the MEF-TOD protocol and the token-passing protocol is shown in Fig. 6 with the corresponding random transfer interval distribution. With the same random interval distribution, the system with the MEF-TOD protocol performs better than the system with the token-passing protocol.

The results of the batch reactor experiment over many runs are presented in Fig. 7. With the same confidence level (probability of passing the performance specifications set at 90% for a fixed average transfer interval time), the maximum average transfer intervals for the system with the token-passing protocol and with the MEF-TOD protocol are 43 ms and 54 ms, respectively.

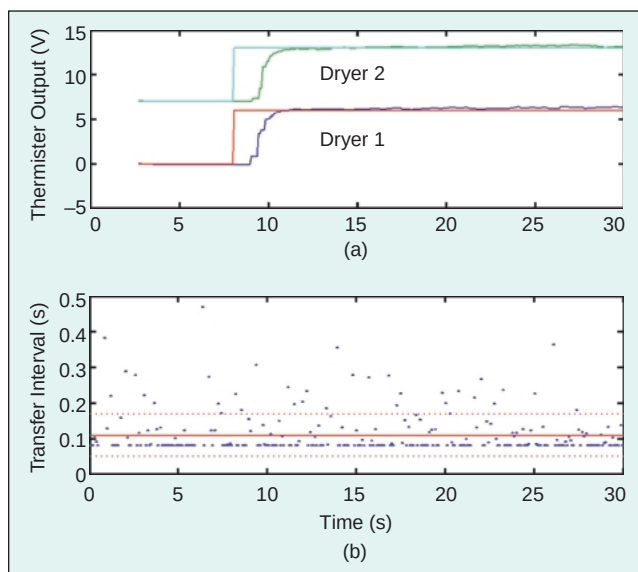


Figure 10. Experimental results using an IEEE 802.11 network. The upper graph shows the commanded and reported step responses of both dryer plants with plant 2 signals shifted by 7 V. The lower graph details the relevant network statistics during the course of the run. (a) Plant commands and reported outputs, dryers 1 and 2; (b) network transfer intervals, mean, and standard deviation.

Dryer Experiment

In the dryer (Feedback's process trainer PT326 shown in Fig. 8), air drawn from the atmosphere by a fan is driven past a heater grid and through a length of tubing to the atmosphere again. The outlet air temperature is measured by a thermistor connected to a smart sensor on the CAN network. The output temperatures are sent over the network to a CAN-enabled computer, which in turn generates the input voltage for the heater grid amplifier. The purpose of the control equipment is to measure the outlet air temperature, compare it with a value set by the operator, and generate a control signal that determines the input u .

The identified plant model with throttle setting at 40% and sampling period 80 ms is shown in the following:

$$G_p(z) = \frac{0.03837 + 0.05656z^{-1}}{1 - 1.37z^{-1} + 0.4516z^{-2}} z^{-4}. \quad (7)$$

Since the plant has a transport delay of about 320 ms, the control law was designed using the Smith predictor combined with a PI controller. We treat the network as transparent when designing the control law. For comparison, the results of a step response test of the PT326 under constant transfer intervals (control transmissions with fixed network access delays) and random transfer intervals (control transmissions with variable network access delays) are illustrated in Fig. 9. For the constant transfer interval, the test reveals the transport delay to be about 300 ms, similar to the estimated delay. Under random transfer intervals, the apparent delay is larger. For example, if the average transfer interval is 80 ms, all intervals smaller than 80 ms are rounded up, so the actual mean transfer interval is approximately 120 ms. Network-induced random time-varying delay can be considered as the measurement delay, since when no new data is available, an old sample is used to update the control law, creating an average measurement delay. This explains why the system behaves well with an average transfer interval of $\tau = 150$ ms and an estimated delay of 400 ms, the approximate sum of the network and plant transport delay.

The results of the dryer experiment validate our former theoretical analysis and assumptions. They also suggest that including the average transport delay into the controller design may improve controller performance. The sampled mean of the random transfer interval can be added to the estimated plant delay in the predictor; however, this method is limited to small average delay intervals and variations. Fig. 10 shows the step responses of two dryer plants sharing a wireless IEEE 802.11 network modified for networked control.

Conclusion

The field of networked control systems is technologically driven and motivated by cost, maintenance, and reliability. The architectural shift does not require reinvention of control theory, but the use of networks will be common in practice, and those building control systems should be familiar with the real-time networking technology available. For researchers, the field of networked control systems has much to offer. In this article, for example, performance gains were demonstrated by dispensing with queues and dynamically scheduling network traffic. Developing techniques to dynamically schedule traffic on the common wireline and wireless networks remains a challenge. The current analysis is very conservative. Although it shows that given sufficient network speed, nothing bad will happen, it does not say much about the average behavior. A detailed analysis would be more useful to design engineers. Patching data between communication events with estimators or linear predictive coding techniques appears promising. Ultimately, research in this area promises the interesting combination of information and control theory.

References

- [1] R. Raji, "Smart network for control," *IEEE Spectrum*, pp. 49-55, June 1994.
- [2] R. Brockett, "Stabilization of motor networks," in *Proc. IEEE Conf. Decision and Control*, New Orleans, LA, Dec. 1995, pp. 1484-1488.
- [3] D. Hristu and K. Morgansen, "Limited communication control," *Syst. Contr. Lett.*, vol. 37, no. 4, pp. 193-205, July 1999.
- [4] D. Hristu, "Optimal control with limited communication," Ph.D. dissertation, Harvard Univ., 1999.
- [5] H. Ye, G. Walsh, and L. Bushnell, "Wireless local area networks in the manufacturing industry," in *Proc. Amer. Control Conf.*, Chicago, IL, June 2000, pp. 2363-2367.
- [6] H.M. Newman, "Integrating building automation and control products using the BACnet protocol," *ASHRAE J.*, vol. 38, no. 11, pp. 36-42, Nov. 1996.
- [7] E. Tovar and F. Vasques, "Real-time Fieldbus communications using Profibus networks," *IEEE Trans. Indust. Electron.*, vol. 46, pp. 1241-1251, Dec. 1999.
- [8] M. Santori, "A tale of three buses: DeviceNet, Profibus-DP, Foundation Fieldbus," *EDN*, vol. 42, no. 22, pp. 149-160, Oct. 1997.
- [9] A. Leach, "Profibus: The German Fieldbus standard," *Assembly Automation*, vol. 14, no. 1, pp. 8-12, 1994.
- [10] A.S. Tanenbaum, *Computer Networks*, 3rd ed. Englewood Cliffs, NJ: Prentice-Hall, 1996.
- [11] A. Ray, "Performance evaluation of medium access control protocols for distributed digital avionics," *ASME J. Dynam. Syst., Measure. Contr.*, vol. 109, no. 4, pp. 370-377, Dec. 1987.
- [12] A. Ray, "Distributed data communication networks for real-time process control," *Chem. Eng. Commun.*, vol. 65, pp. 139-154, Mar. 1988.
- [13] J. Nilsson, B. Bernhardsson, and B. Wittenmark, "Stochastic analysis of control of real-time systems with random time delays," *Automatica*, vol. 34, no. 1, pp. 57-64, Jan. 1998.
- [14] W.S. Wong and R.W. Brockett, "Systems with finite communication bandwidth constraints, part I: State estimation problems," *IEEE Trans. Automat. Contr.*, vol. 42, pp. 1294-1299, Sept. 1997.
- [15] W.S. Wong and R.W. Brockett, "Systems with finite communication bandwidth constraints, part II: Stabilization with limited information feedback," *IEEE Trans. Automat. Contr.*, vol. 44, pp. 1049-1053, May 1999.
- [16] G. Walsh, H. Ye, and L. Bushnell, "Stability analysis of networked control systems," in *Proc. Amer. Control Conf.*, San Diego, CA, June 1999, pp. 2876-2880.
- [17] G. Walsh, O. Beldiman, and L. Bushnell, "Asymptotic behavior of networked control systems," in *Proc. IEEE Int. Conf. Control and Applications*, Hawaii, 1999, pp. 1448-1454.
- [18] G. Walsh, O. Beldiman, and L. Bushnell, "Error encoding algorithms for networked control systems," in *Proc. IEEE Conf. Decision and Control*, Phoenix, AZ, Dec. 1999, pp. 4933-4938.
- [19] J.D. Decotignie and D. Auslander, "Integrated communication and control systems with fieldbuses," in *Japan/USA Symp. Flexible Automation*, vol. 1, 1996, pp. 517-520.
- [20] K.G. Shin, "Real-time communications in a computer-controlled workcell," *IEEE Trans. Robotics and Automation*, vol. 7, pp. 105-113, Feb. 1991.
- [21] H. Zeltwanger, "An inside look at the fundamentals of CAN," *Contr. Eng.*, pp. 81-87, Jan. 1995.
- [22] K.M. Zuberi and K.G. Shin, "Scheduling messages on controller area network for real-time CIM applications," *IEEE Trans. Robot. Automat.*, vol. 13, pp. 310-314, Apr. 1997.
- [23] H. Khalil, *Nonlinear Systems*, 2nd ed. Englewood Cliffs, NJ: Prentice-Hall, 1996.
- [24] M. Green and D. Limebeer, *Linear Robust Control*. Englewood Cliffs, NJ: Prentice-Hall, 1995.

Gregory C. Walsh received the B.S. degree in electrical engineering from the University of Maryland in 1989. From the University of California at Berkeley, he received an M.S. and a Ph.D. in electrical engineering and computer sciences in 1990 and 1994, respectively. From the same university he also earned an M.A. in mathematics in 1994. From 1994 to 1995 he was a post-doctoral researcher at the Institute for Systems Research at the University of Maryland, and from 1995 to the present, an Assistant Professor of Mechanical Engineering jointly with the Institute for Systems Research at the University of Maryland. His research interests range from nonlinear to networked control, with applications in process control, magnetic suspension systems, and hybrid electric vehicles.

Hong Ye received the B.Sc. degree in automation from Tsinghua University, Beijing, China, in 1992 and the M.Sc. degree in control and fluid transmission from the First Academy of China National Aerospace, Beijing, China, in 1995. In 2000, she received the Ph.D. degree in mechanical engineering from the University of Maryland, College Park. Since September, 2000, she has been with Delphi Communication Systems, Maynard, MA. Her research interests are in third-generation wireless communication systems, digital signal processing, and the application of networked control systems. She is a member of the IEEE.